RAPPORT DE PROJET

SecWatch

Application de déclaration et de suivi des incidents

Sécurité des systèmes distribués

Spring Boot React JWT

Étudiant : M'barek ETTALEBY
Formation : SDIA2
Enseignant : Abdelmajid BOUSSELHAM
Établissement : ENSETM
Année universitaire : 2026–2027

Résumé

SecWatch est une application web de déclaration et de suivi des incidents informatiques ou de cybersécurité. Elle distingue trois rôles : EMPLOYEE, TECHNICIAN et ADMIN. Son backend Spring Boot expose une API REST sécurisée par Spring Security et des jetons JWT. Son frontend React adapte les pages au rôle connecté et communique avec l'API au moyen d'Axios.

L'application permet de créer et suivre des incidents, de les affecter à un technicien, de modifier leur statut, d'ajouter des notes et de consulter l'historique. Elle comprend aussi les profils, l'administration des utilisateurs, les validations, les erreurs API et les tableaux de bord par rôle.

Mots-clés : JWT, Spring Security, React, JPA, authentification, contrôle d'accès, cybersécurité.

Table des matières

1 Présentation du projet **1**

1.1 Contexte 1

1.2 Objectifs 1

1.3 Rôles 1

1.4 Périmètre 2

2 Architecture et organisation **3**

2.1 Architecture générale 3

2.2 Organisation du dépôt 3

2.3 Modèle de données 3

3 Modèle de données et règles métier **5**

3.1 Utilisateur 5

3.2 Incident 5

3.3 Historique et commentaires 5

3.4 Règles métier 5

4 Authentification et sécurité **7**

4.1 Inscription et connexion 7

4.2 JWT 7

4.3 Filtre JWT et sessions sans état 8

4.4 Autorisations par rôle et propriétaire 8

4.5 Expiration et désactivation 9

5 API REST **10**

5.1 Endpoints principaux 10

5.2 Validation et codes HTTP 10

6 Application React **12**

6.1 Routes et organisation 12

6.2 Axios et stockage du JWT 12

6.3 Tableaux de bord 13

7 Installation et démonstration **14**

7.1 Prérequis 14

7.2 Démarrage du backend 14

7.3 Démarrage du frontend 14

7.4 Comptes de démonstration 14

7.5 Scénario fonctionnel 15

8 Vérifications de sécurité avec curl **16**

8.1 Cas requis 16

8.2 Résultats des vérifications 16

9 Captures de l'application **22**

10 Résultats, limites et conclusion **27**

10.1 Résultats 27

10.2 Limites connues 27

10.3 Conclusion 27

Chapitre 1

Présentation du projet

1.1 Contexte

Les entreprises dépendent de leurs postes de travail, de leurs applications et de leurs réseaux. Une infection par logiciel malveillant, une tentative d'hameçonnage, une panne réseau, un problème d'accès ou une fuite potentielle de données doit pouvoir être signalé, attribué et suivi jusqu'à sa résolution.

SecWatch centralise ces signalements dans deux projets distincts : un backend Spring Boot et un frontend React. Les données sont persistées avec JPA. H2 est utilisé par défaut en développement ; MySQL peut être configuré avec les variables d'environnement.

1.2 Objectifs

— authentifier les utilisateurs avec un JWT à durée limitée ;

— contrôler les accès selon les rôles et la propriété des incidents ;

— valider les données et produire des erreurs HTTP cohérentes ;

— créer, affecter et suivre les incidents ;

— conserver l'historique et les notes de traitement ;

— fournir des tableaux de bord adaptés aux rôles.

1.3 Rôles

Rôle	Responsabilités
EMPLOYEE	Consulter et modifier son profil, créer des incidents et consulter uniquement ses propres signalements.
TECHNICIAN	Consulter les incidents affectés, modifier leur statut et ajouter des notes de traitement.

ADMIN	Consulter les incidents et utilisateurs, affecter les techniciens, activer ou désactiver les comptes et consulter les statistiques.

1.4 Périmètre

L'inscription publique attribue toujours le rôle EMPLOYEE. Les comptes ADMIN et TECHNICIAN de démonstration sont préchargés. Le Refresh Token est facultatif dans le sujet et n'est pas implémenté ; le frontend demande une nouvelle connexion lorsque l'access token expire.

Chapitre 2

Architecture et organisation

2.1 Architecture générale

Le navigateur charge React. Axios envoie les requêtes HTTP et ajoute le JWT à l'en-tête `Authorization`. Spring Security filtre les requêtes avant les contrôleurs. Les contrôleurs valident les entrées, les services appliquent les règles métier et les repositories JPA communiquent avec la base.

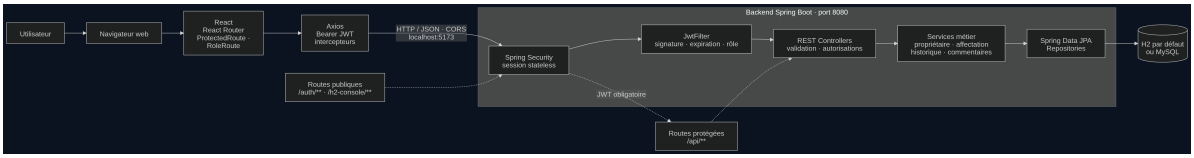


FIGURE 2.1 – Architecture logique de SecWatch

2.2 Organisation du dépôt

— `backend/` : modèles JPA, contrôleurs, sécurité, services et repositories ;

— `frontend/` : composants React, routes, pages et services Axios ;

— `docs/` : rapport et diagrammes Mermaid/SVG ;

— `postman/` : collection de requêtes de démonstration.

2.3 Modèle de données

Un incident possède un créateur obligatoire et peut être affecté à un technicien. L'historique et les commentaires sont des entités séparées : un incident peut recevoir plusieurs événements et notes. Les relations utilisent principalement `@ManyToOne`. Le chargement différé limite les chargements inutiles ; les requêtes chargent les références nécessaires avant de construire les DTOs.

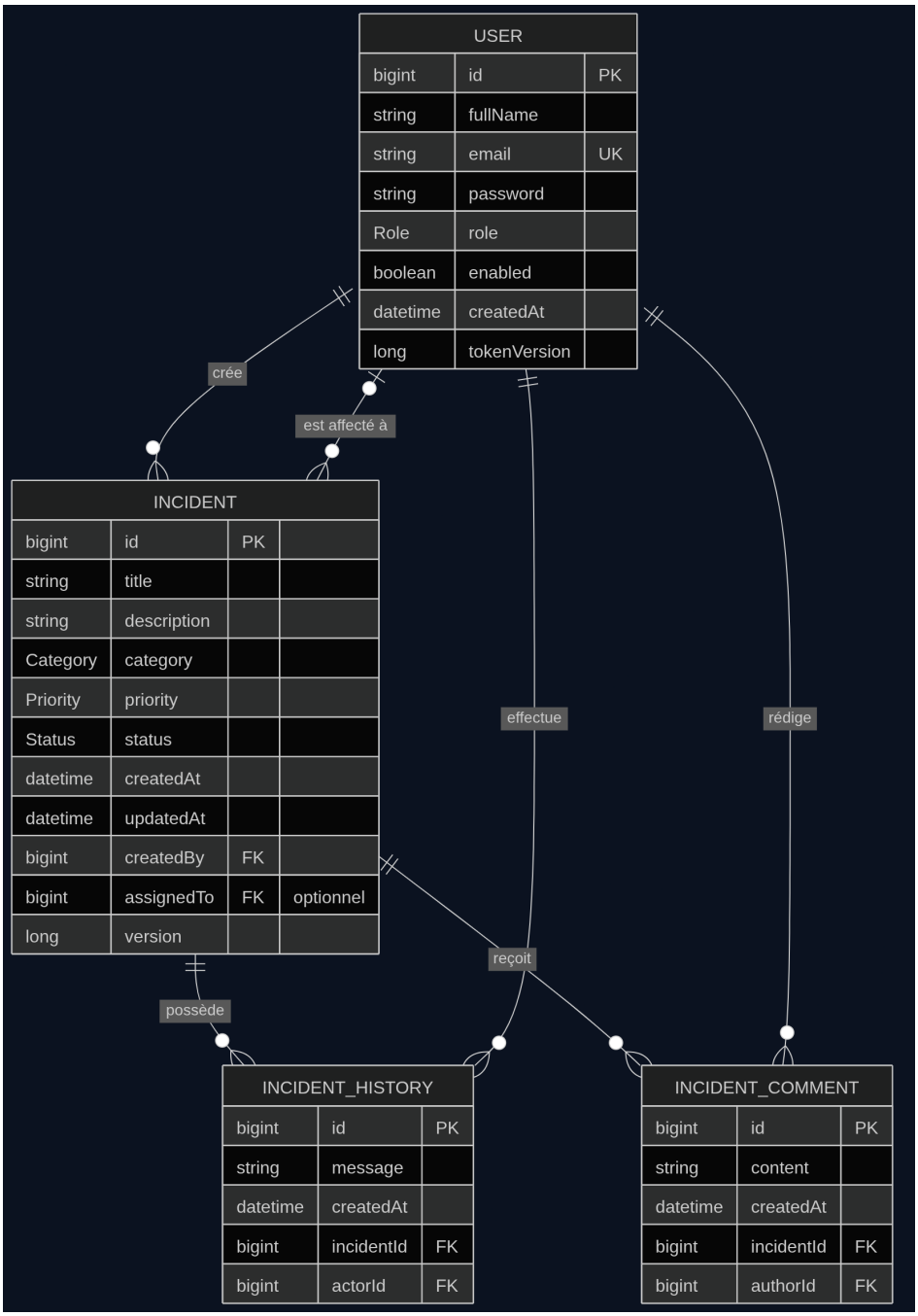


FIGURE 2.2 – Entités principales et relations de SecWatch

Les incidents référencent leur créateur et, facultativement, le technicien affecté. Chaque événement et chaque note est lié à l'incident ainsi qu'à l'utilisateur qui l'a enregistré.

Les diagrammes intégrés au rapport sont les fichiers PNG `diagram-entities.png` et `diagram-architecture.png`.

Chapitre 3

Modèle de données et règles métier

3.1 Utilisateur

L'entité `User` contient l'identifiant, le nom complet, l'email, le mot de passe encodé, le rôle, l'état actif, la date de création et la version du token. L'email est unique. `tokenVersion` invalide les JWT émis avant une désactivation ou un changement de mot de passe.

3.2 Incident

L'entité `Incident` contient le titre, la description, la catégorie, la priorité, le statut, les dates de création et de mise à jour, le créateur, le technicien facultatif et un champ `@Version` pour le verrouillage optimiste.

Champ	Valeurs
Catégorie	NETWORK, MALWARE, PHISHING, ACCESS, SOFTWARE, HARDWARE, DATA_LEAK, OTHER
Priorité	LOW, MEDIUM, HIGH, CRITICAL
Statut	OPEN, ASSIGNED, IN_PROGRESS, RESOLVED, CLOSED

3.3 Historique et commentaires

Chaque action importante est enregistrée dans `IncidentHistory` avec son acteur, son message et son horodatage. Les affectations et changements de statut créent un événement. `IncidentComment` conserve les notes de traitement. Le détail affiche les événements par ordre chronologique.

3.4 Règles métier

1. Une inscription publique attribue EMPLOYEE ; le client ne choisit jamais son rôle.

2. Les mots de passe sont encodés avec BCrypt.
3. Un employé ne consulte que ses propres incidents.
4. Un technicien ne traite que les incidents qui lui sont affectés.
5. Seul ADMIN affecte un technicien et consulte les données globales.
6. Un compte désactivé ne peut plus se connecter.
7. Un incident CLOSED ne change plus d'état et ne reçoit plus de note.

Chapitre 4

Authentification et sécurité

4.1 Inscription et connexion

Les routes publiques `POST /auth/register` et `POST /auth/login` reçoivent des DTOs validés. Le DTO d'inscription ne contient pas de rôle. Le service encode le mot de passe avec BCrypt et affecte lui-même le rôle EMPLOYEE. La réponse de connexion ne contient aucun mot de passe.

Listing 4.1 – DTO d'inscription et rôle fixé côté serveur

```
public record Register(
    @NotBlank @Size(max = 100) String fullName,
    @NotBlank @Email @Size(max = 254) String email,
    @NotBlank @Size(min = 8, max = 72) String password
) {}

// Dans le service
user.password = passwordEncoder.encode(request.password());
user.role = Role.EMPLOYEE;
users.save(user);
```

4.2 JWT

Le JWT est signé en HMAC. Il contient l'email comme sujet, les dates d'émission et d'expiration, le rôle, l'identifiant utilisateur et la version du compte. Sa durée par défaut est de 300 secondes. Le parseur vérifie la signature et l'expiration avant de rendre les claims disponibles.

Listing 4.2 – Claims inclus dans le JWT

```
return Jwts.builder()
    .subject(user.email)
    .issuedAt(Date.from(now))
    .expiration(Date.from(now.plusSeconds(ttl)))
    .claim("role", user.role.name())
    .claim("uid", user.id)
    .claim("ver", user.tokenVersion)
```

```
8       .signWith(key)
9       .compact();
```

Un claim est une donnée portée par le token. `role` sert aux autorisations Spring, `uid` retrouve le compte même si l'email change et `ver` invalide les tokens antérieurs à une révocation.

4.3 Filtre JWT et sessions sans état

`JwtFilter` lit l'en-tête `Authorization: Bearer ...`, vérifie le token, recharge l'utilisateur, vérifie son activation et compare la version du JWT à celle enregistrée. Il place ensuite l'authentification dans le `SecurityContext`. `SessionCreationPolicy.STATELESS` indique que le serveur ne conserve pas de session HTTP.

Listing 4.3 – Extrait de la configuration Spring Security

```
1 http.csrf(csrf -> csrf.disable())
2     .sessionManagement(session -> session
3         .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
4     .authorizeHttpRequests(auth -> auth
5         .requestMatchers("/auth/**", "/h2-console/**").permitAll()
6         .requestMatchers("/api/**").authenticated()
7         .anyRequest().permitAll());
```

Le CORS autorise le frontend, par défaut `http://localhost:5173`, ainsi que les méthodes et en-têtes nécessaires à Axios.

4.4 Autorisations par rôle et propriétaire

Les annotations `@PreAuthorize` contrôlent les rôles au niveau des endpoints. Le service vérifie également que l'utilisateur est le propriétaire de l'incident ou le technicien affecté. Le masquage d'un bouton React n'est pas une protection serveur.

Listing 4.4 – Contrôle d'accès sur un incident

```
1 boolean canSee(Incident incident, User user) {
2     return user.role == Role.ADMIN
3         || sameUser(incident.createdBy, user)
4         || (user.role == Role.TECHNICIAN
5             && sameUser(incident.assignedTo, user));
6 }
```

Les relations JPA différées sont lues par des getters ou des requêtes qui chargent les références nécessaires. Cela évite l'accès direct aux champs d'un proxy Hibernate non initialisé.

4.5 Expiration et désactivation

Quand une route protégée répond 401, Axios supprime le token et le profil local, puis redirige vers la connexion en affichant un message de session expirée. Un compte désactivé ne peut pas s'authentifier et ses anciens tokens sont rejetés. Le Refresh Token est facultatif et n'est pas inclus.

Chapitre 5

API REST

5.1 Endpoints principaux

Méthode	Endpoint	Accès
POST	`/auth/register`, `/auth/login`	Public
GET / PUT	`/api/profile`	Authentifié
PUT	`/api/profile/password`	Authentifié
POST	`/api/incidents`	EMPLOYEE
GET	`/api/incidents/my`	EMPLOYEE
GET	`/api/incidents/{id}`	Propriétaire, technicien affecté ou ADMIN
GET	`/api/incidents/assigned`	TECHNICIAN
GET	`/api/incidents`	ADMIN
PUT	`/api/incidents/{id}/status`	Technicien affecté ou ADMIN
PUT	`/api/incidents/{id}/assign/{technicianId}`	ADMIN
POST	`/api/incidents/{id}/comments`	Technicien affecté ou ADMIN
GET	`/api/admin/users`	ADMIN
PUT	`/api/admin/users/{id}/enable` ou `/disable`	ADMIN
GET	`/api/admin/statistics`	ADMIN
GET	`/api/admin/dashboard`	ADMIN ; statistiques et incidents

5.2 Validation et codes HTTP

Les DTOs utilisent `@NotBlank`, `@Email`, `@Size` et `@NotNull` ; les contrôleurs déclenchent la validation avec `@Valid`. `@ControllerAdvice` transforme les exceptions en réponses JSON contenant le code HTTP et un message.

Code	Signification dans l'application

400	Données invalides, ancien mot de passe incorrect ou confirmation différente.
401	JWT absent, invalide ou expiré ; identifiants incorrects ; compte désactivé.
403	Rôle insuffisant ou tentative d'accès à l'incident d'un autre employé.
404	Incident, utilisateur ou technicien introuvable.
409	Adresse email déjà utilisée ou conflit métier.

Chapitre 6

Application React

6.1 Routes et organisation

`ProtectedRoute` vérifie la présence d'un token et `RoleRoute` limite certaines pages aux rôles autorisés. Ces composants guident l'utilisateur ; l'API effectue le contrôle d'accès réel. Les pages couvrent la connexion, l'inscription, les dashboards, le profil, les incidents, les affectations et l'administration.

6.2 Axios et stockage du JWT

Après connexion, le client conserve le token dans le navigateur. L'intercepteur Axios l'ajoute aux requêtes protégées. Le gestionnaire de réponse traite l'expiration :

Listing 6.1 – Intercepteurs Axios

```
api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = "Bearer " + token;
  return config;
});

api.interceptors.response.use((response) => response, (error) => {
  if (error.response?.status === 401 && localStorage.getItem("token")) {
    localStorage.removeItem("token");
    localStorage.removeItem("user");
    sessionStorage.setItem("session-expired", "1");
    window.location.href = "/login";
  }
  return Promise.reject(error);
});
```

6.3 Tableaux de bord

EMPLOYEE
: Nombre d'incidents personnels, ouverts, en cours, résolus ou clôturés, et signalements récents.

TECHNICIAN
: Incidents affectés, priorités critiques, incidents en cours et résolus.

ADMIN
: Utilisateurs, employés, techniciens, incidents par statut, incidents critiques et tableau des incidents récents.

Le dashboard administrateur reçoit ses compteurs et ses lignes depuis un endpoint unique. Le frontend vérifie que le nombre reçu correspond au total annoncé. L'interface utilise une palette zinc et teal, un thème clair/sombre mémorisé localement et des composants inspirés de shadcn/ui. Les fichiers frontend sont formatés avec Prettier.

Chapitre 7

Installation et démonstration

7.1 Prérequis

Java 21, Maven 3.9 ou supérieur, Node.js et npm.

7.2 Démarrage du backend

Listing 7.1 – Lancer Spring Boot

```
cd backend
mvn spring-boot:run
```

L'API écoute sur `http://localhost:8080`. La console H2 est disponible à `http://localhost:8080/h2`
URL JDBC : `jdbc:h2:file:./.data/secwatch` ; utilisateur `sa`, mot de passe vide.

7.3 Démarrage du frontend

Listing 7.2 – Lancer React

```
cd frontend
npm install
npm run dev
```

Vite affiche l'adresse locale, normalement `http://localhost:5173`. Si le port est occupé, ouvrir l'adresse réellement affichée.

7.4 Comptes de démonstration

Les comptes sont préchargés si `SEED_DEMO=true`. Mot de passe local commun : `Demo1234!`.

Rôle	Email
ADMIN	`admin@secwatch.local`
TECHNICIAN	`tech@secwatch.local`
EMPLOYEE	`employee@secwatch.local`
EMPLOYEE	`employee2@secwatch.local`

7.5 Scénario fonctionnel

1. Se connecter comme premier employé et créer deux incidents.
2. Se connecter comme second employé et créer un incident.
3. Vérifier que le premier employé ne peut pas lire l'incident du second.
4. En ADMIN, consulter les incidents et affecter l'un d'eux au technicien.
5. En TECHNICIAN, passer l'incident à IN_PROGRESS, ajouter une note puis le passer à RESOLVED.
6. Revenir au compte employé et vérifier le statut et l'historique.

Chapitre 8

Vérifications de sécurité avec curl

Les neuf situations ci-dessous couvrent les principaux contrôles d'accès et de validation de l'API.

8.1 Cas requis

N°	Test	Requête / préparation	Attendu
1	API sans JWT	GET `/api/profile`, sans Authorization	401
2	JWT invalide	GET profil avec `Bearer invalide`	401
3	JWT expiré	Rejouer le token après expiration	401
4	Employé sur route ADMIN	GET `/api/admin/users` avec token employé	403
5	Incident d'un autre employé	GET `/api/incidents/ID` avec mauvais propriétaire	403
6	Technicien non affecté	PUT statut d'un incident non affecté	403
7	Compte désactivé	POST login après désactivation	401
8	Email déjà utilisé	Répéter l'inscription avec le même email	409
9	Incident invalide	POST avec titre vide ou priorité absente	400

8.2 Résultats des vérifications

Les essais couvrent l'absence et l'invalidité du JWT, l'expiration, les rôles, la propriété des incidents, l'affectation, la désactivation, les doublons et la validation des données. Les réponses observées sont illustrées ci-dessous.

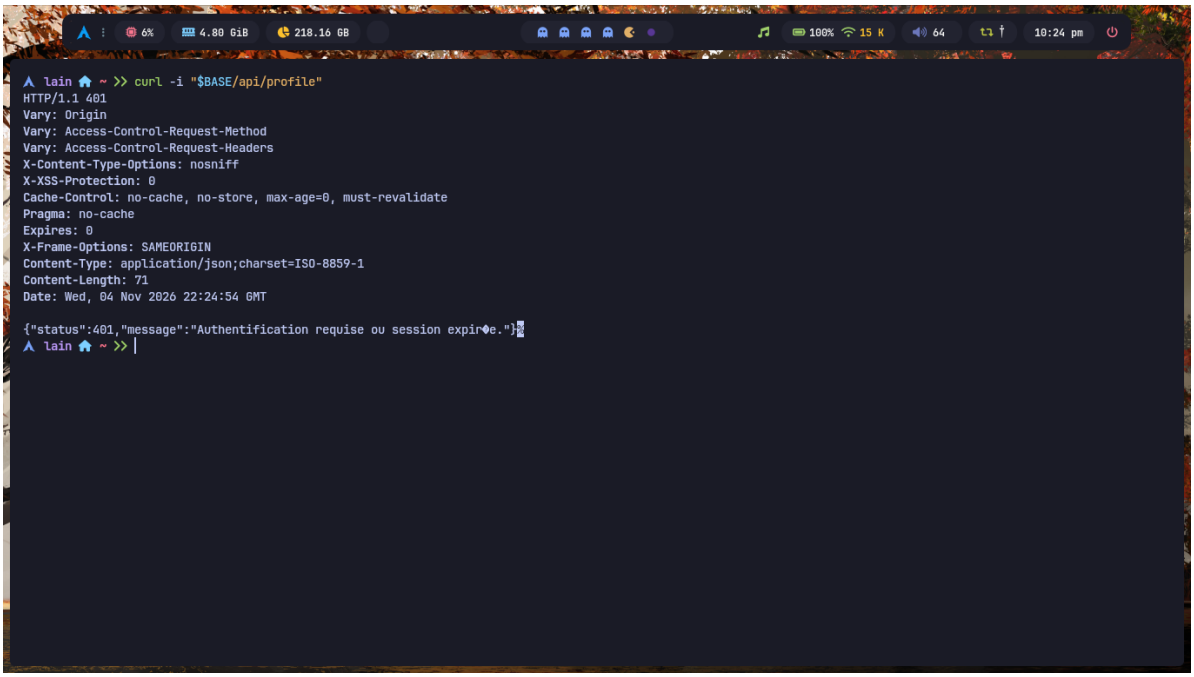


FIGURE 8.1 – API sans JWT — réponse 401

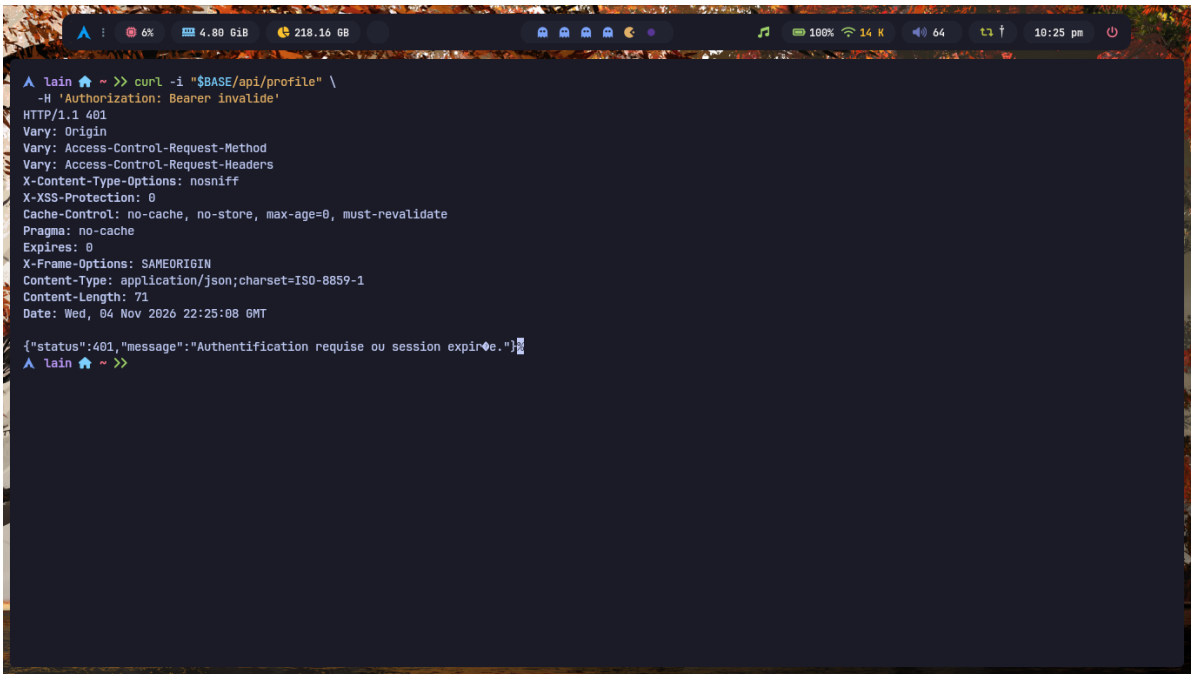


FIGURE 8.2 – JWT invalide — réponse 401

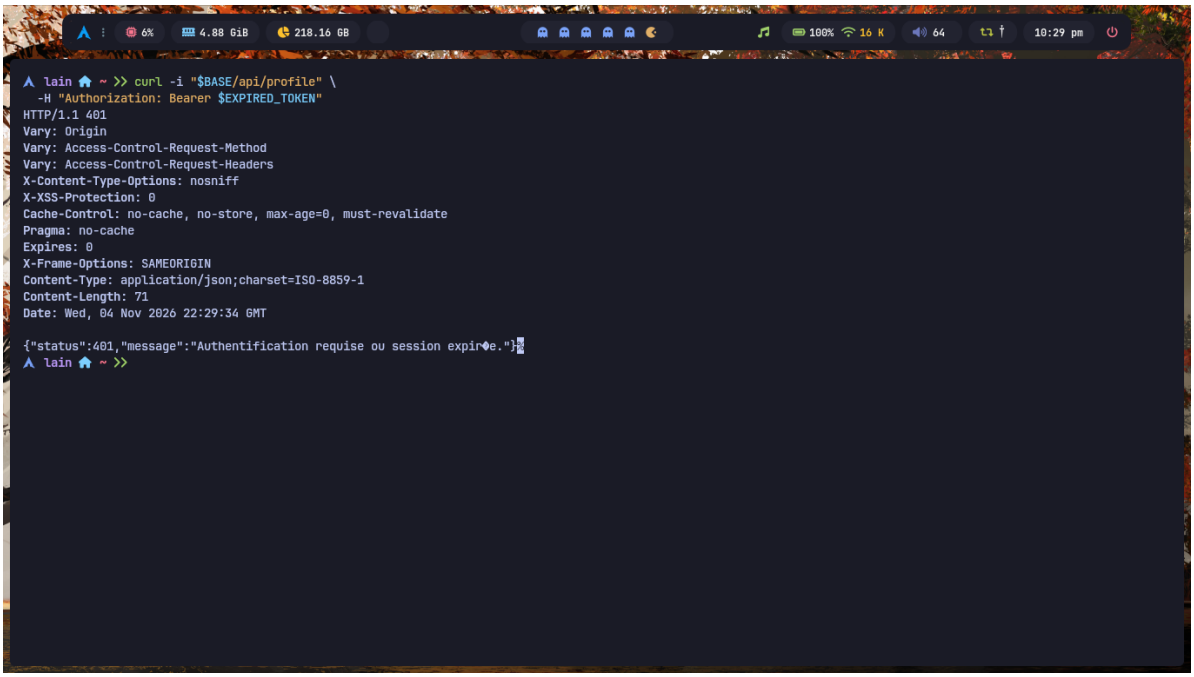


FIGURE 8.3 – JWT expiré — réponse 401

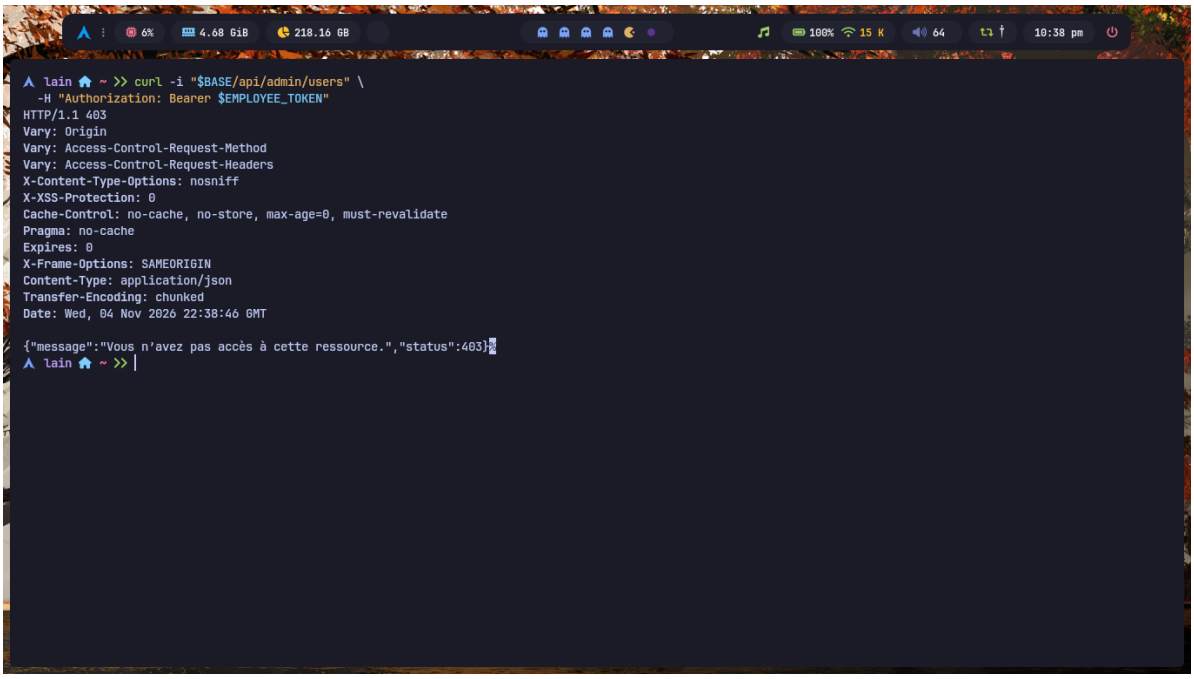


FIGURE 8.4 – Accès employé à une route administrateur — réponse 403

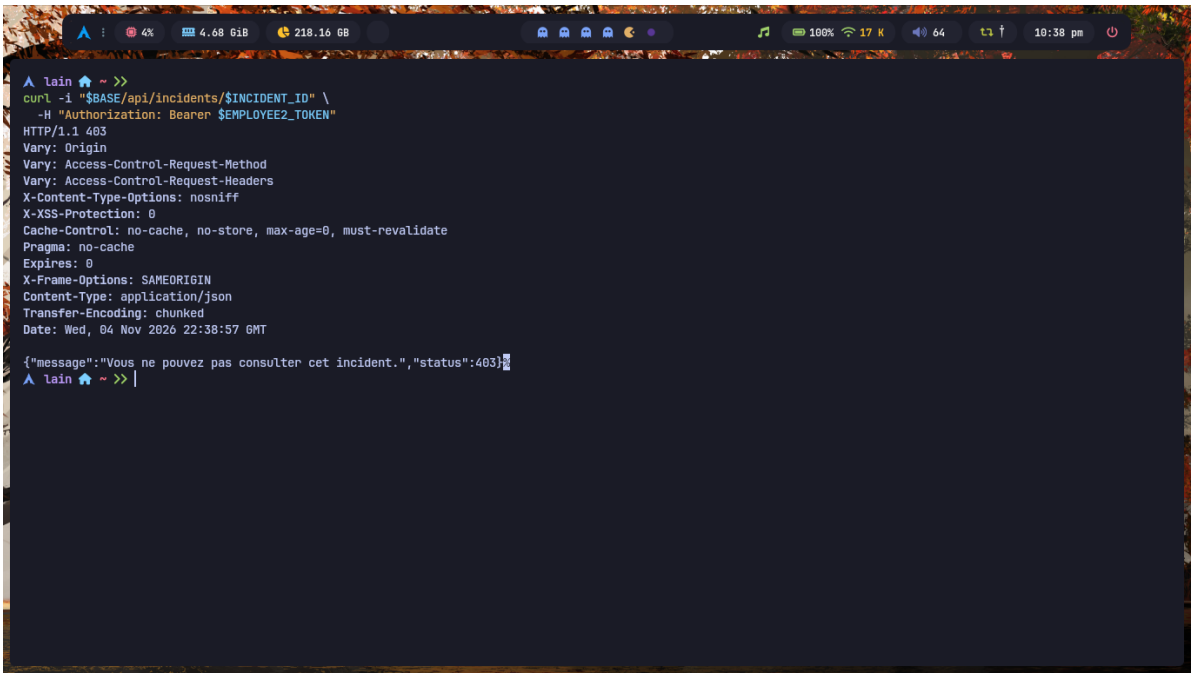


FIGURE 8.5 – Accès à l'incident d'un autre employé — réponse 403

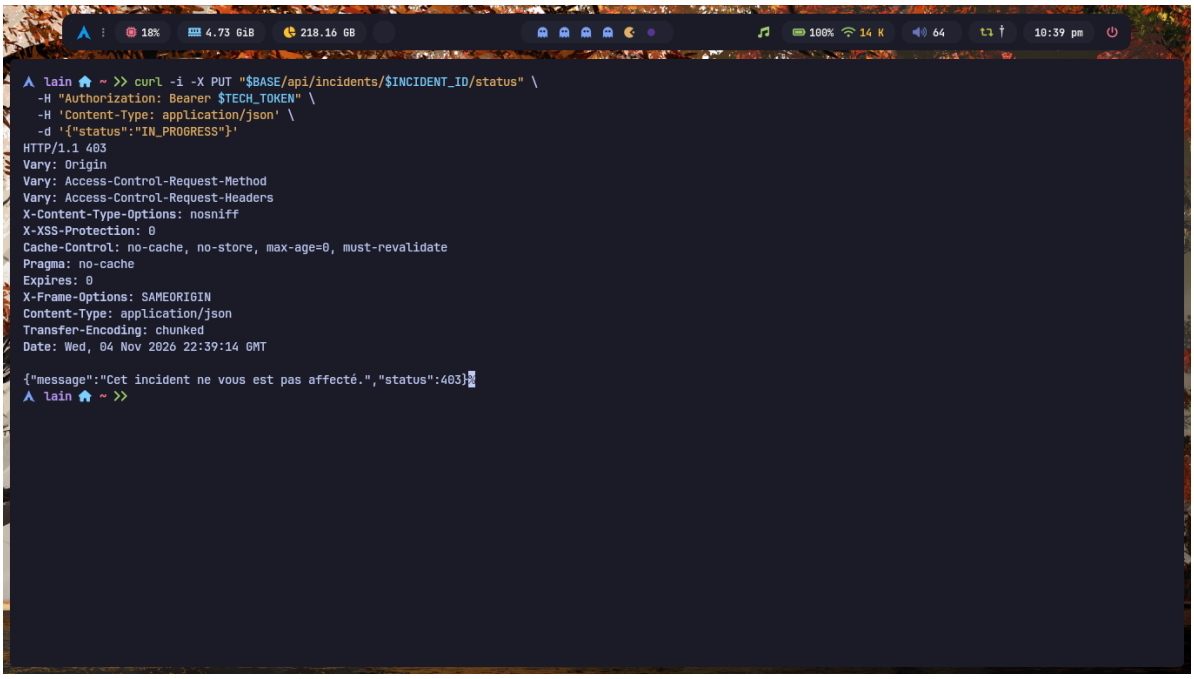


FIGURE 8.6 – Technicien non affecté — réponse 403



FIGURE 8.7 – Connexion d'un compte désactivé — réponse 401

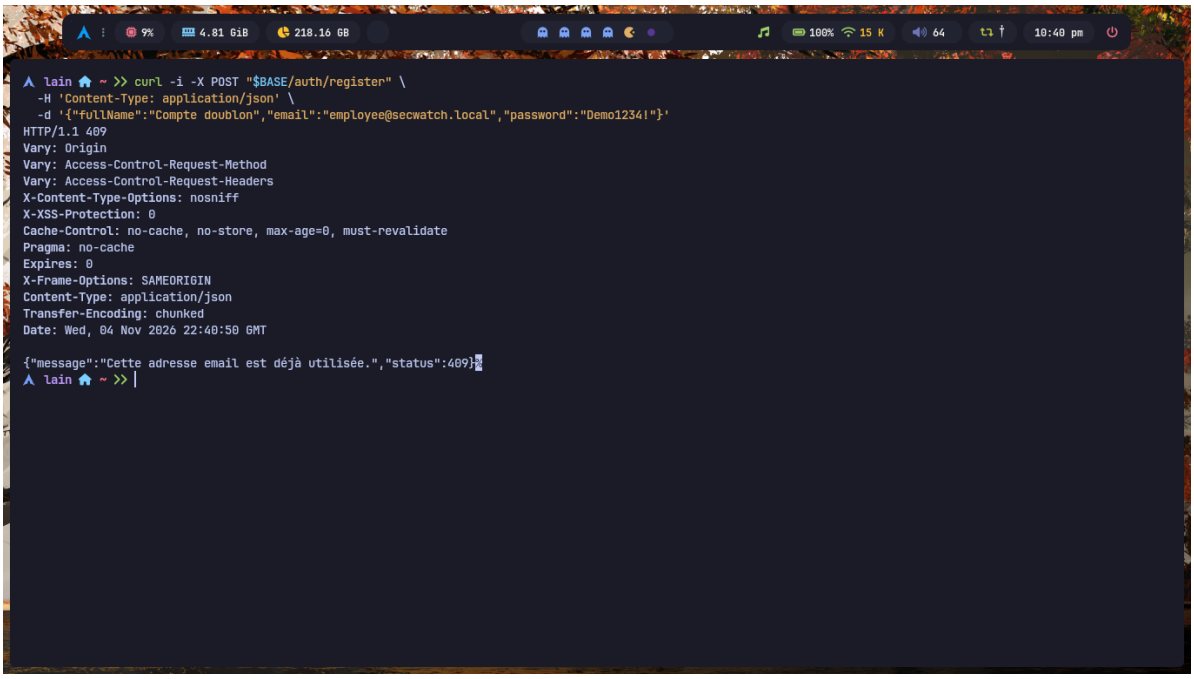


FIGURE 8.8 – Inscription avec email déjà utilisé — réponse 409

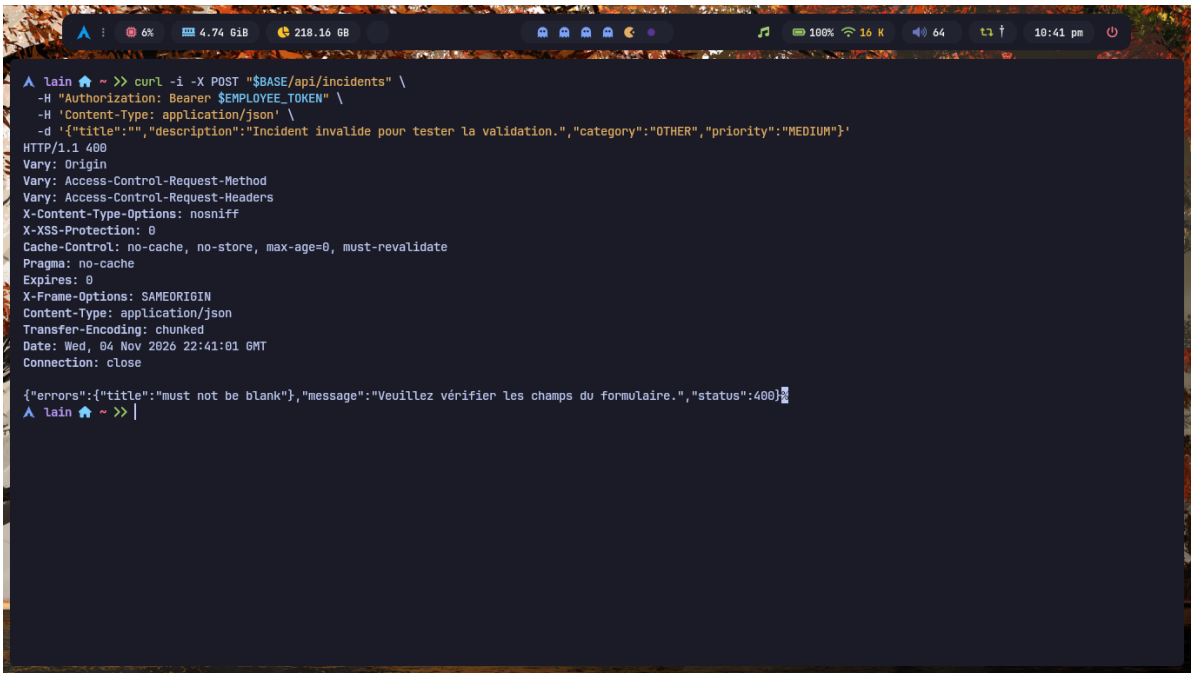


FIGURE 8.9 – Incident invalide — réponse 400

Chapitre 9

Captures de l'application

Les figures suivantes présentent les écrans principaux de SecWatch.

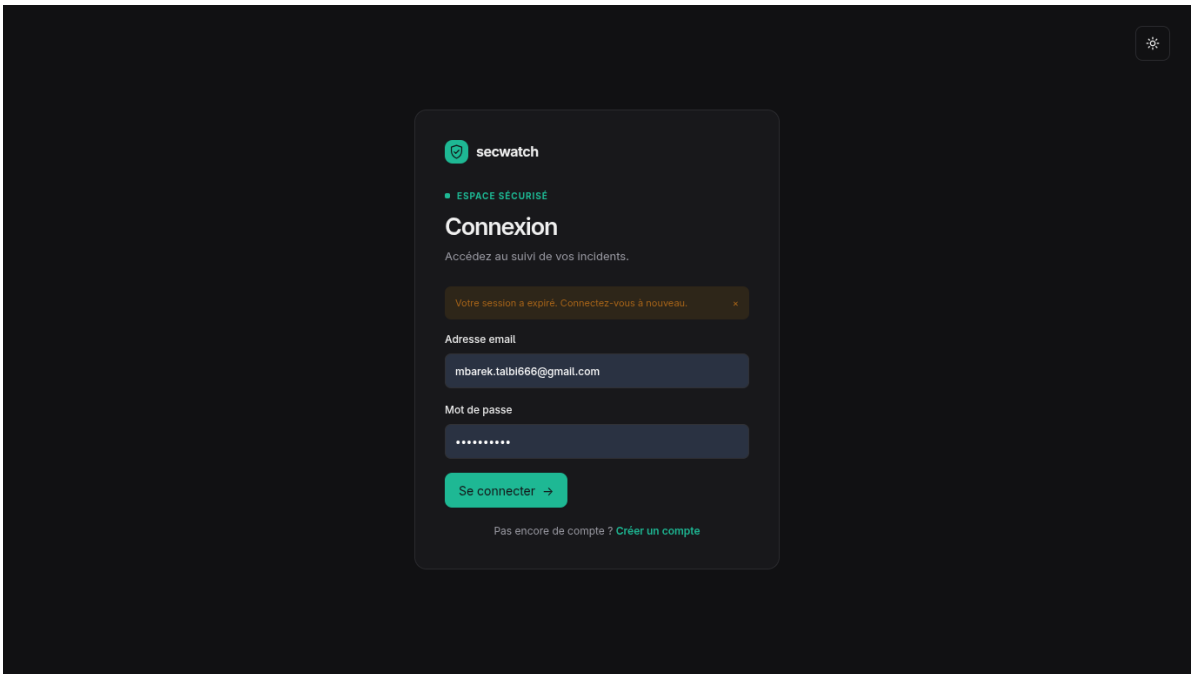


FIGURE 9.1 – Connexion

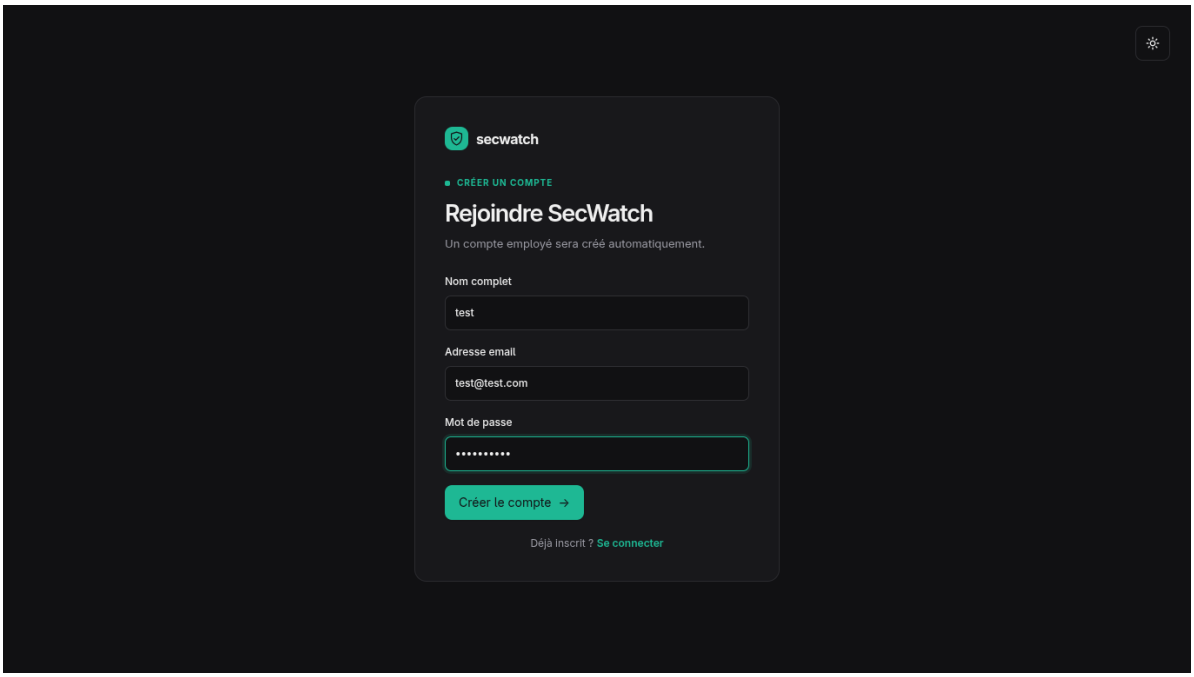


FIGURE 9.2 – Inscription

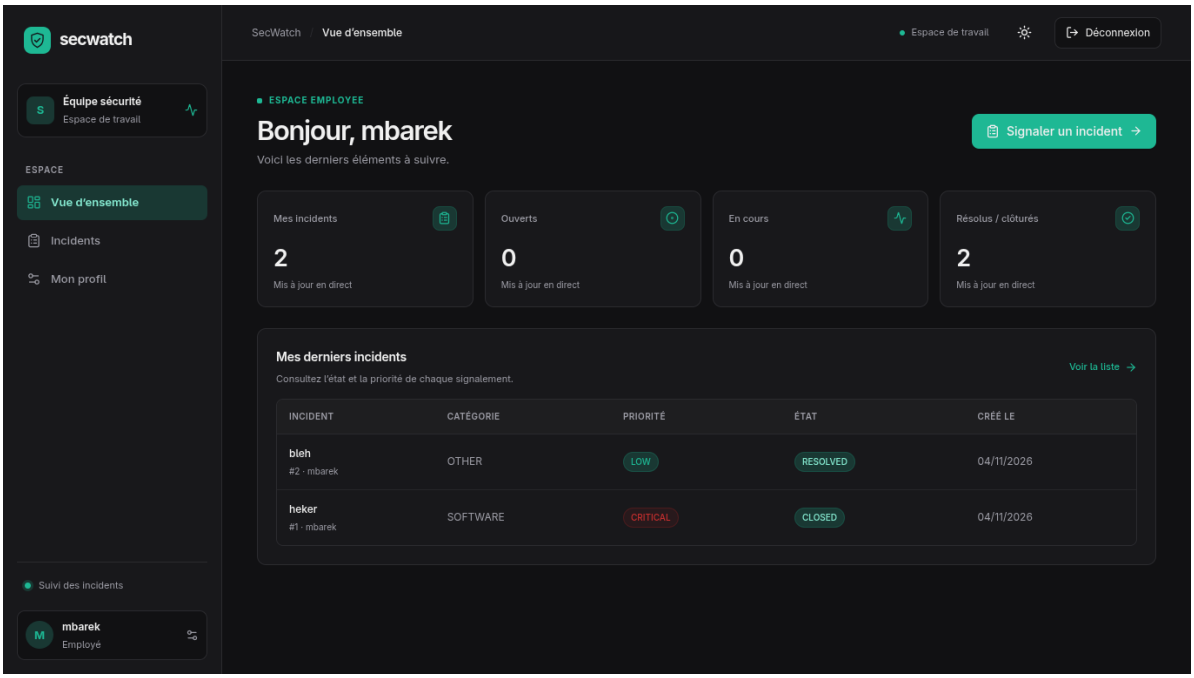


FIGURE 9.3 – Dashboard employé

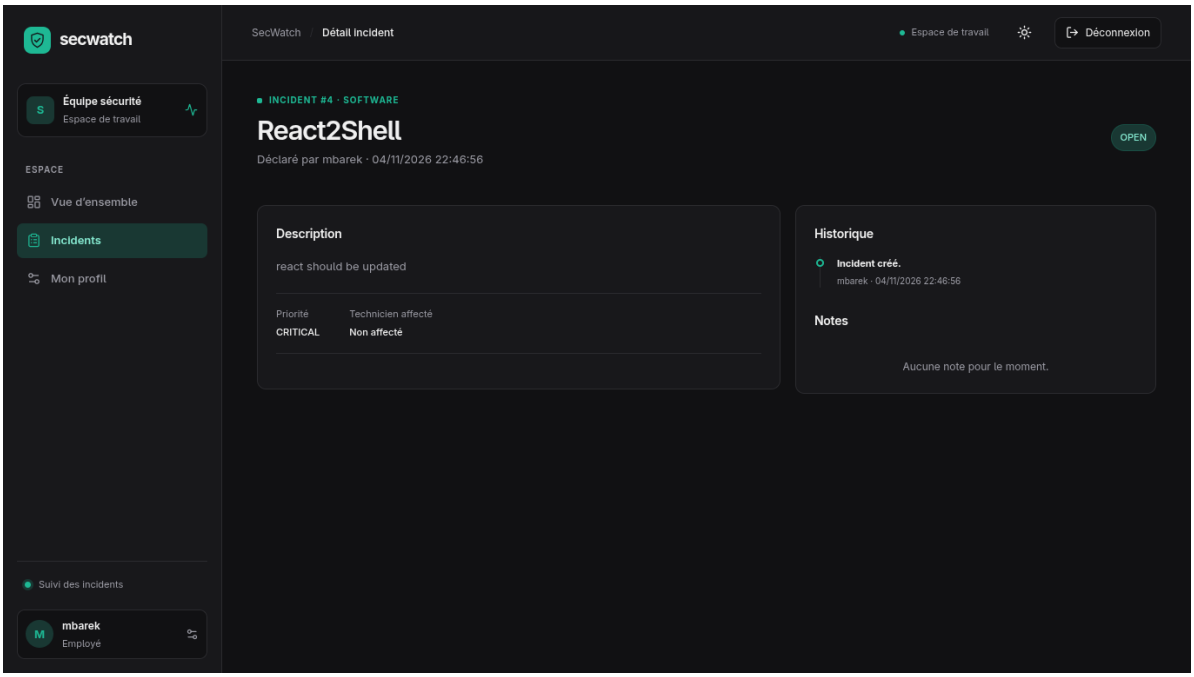


FIGURE 9.4 – Détail d'un incident côté employé

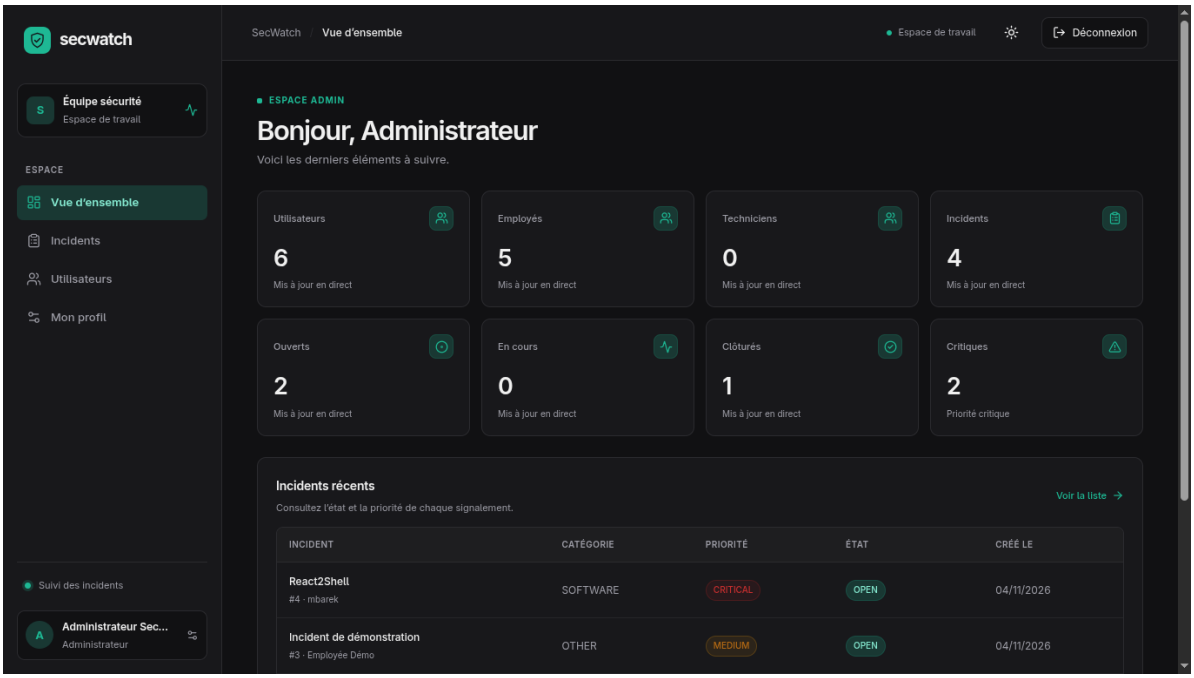


FIGURE 9.5 – Dashboard administrateur

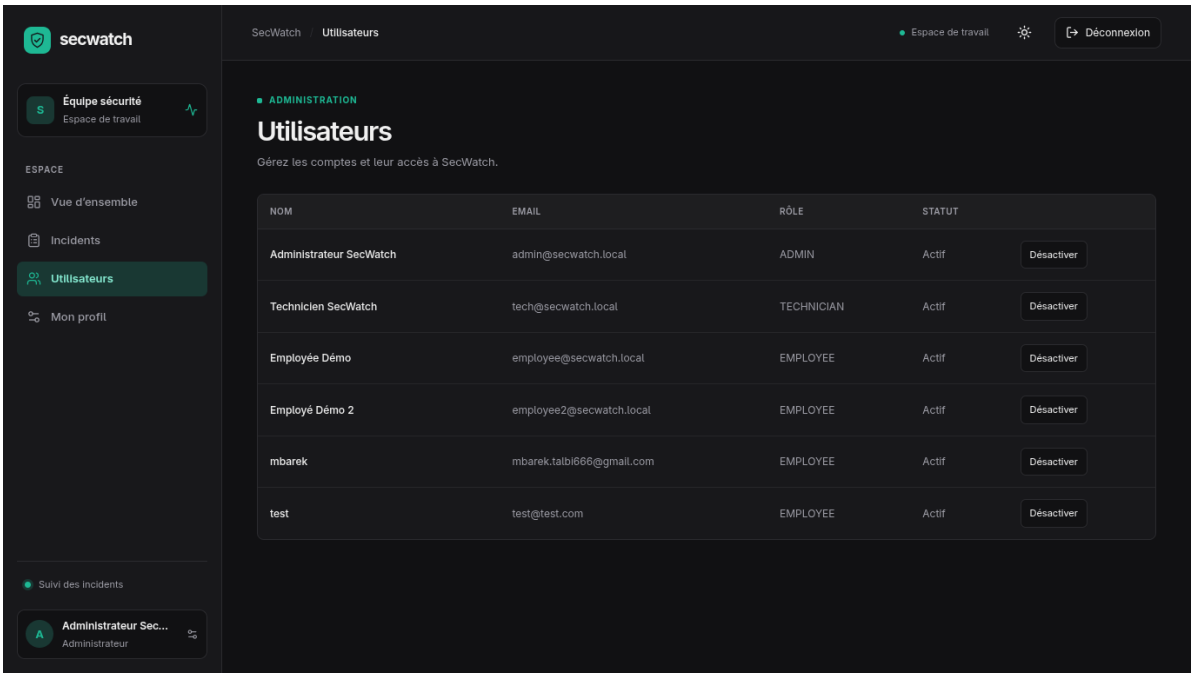


FIGURE 9.6 – Gestion des utilisateurs

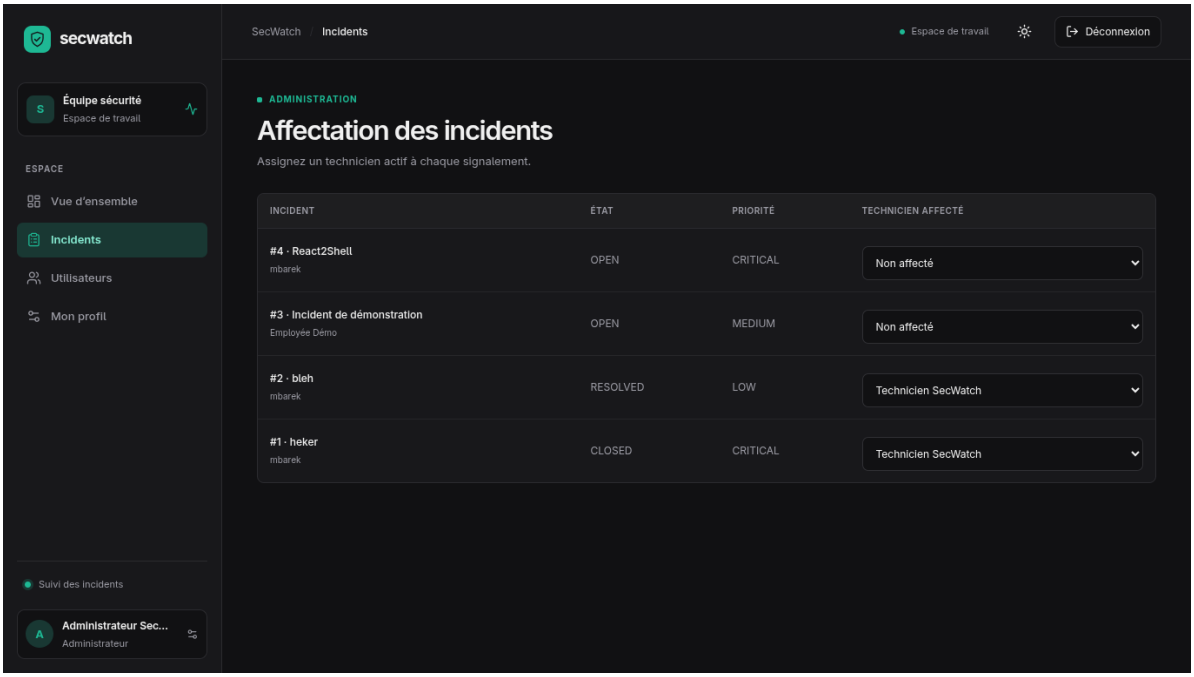


FIGURE 9.7 – Incidents affectés au technicien

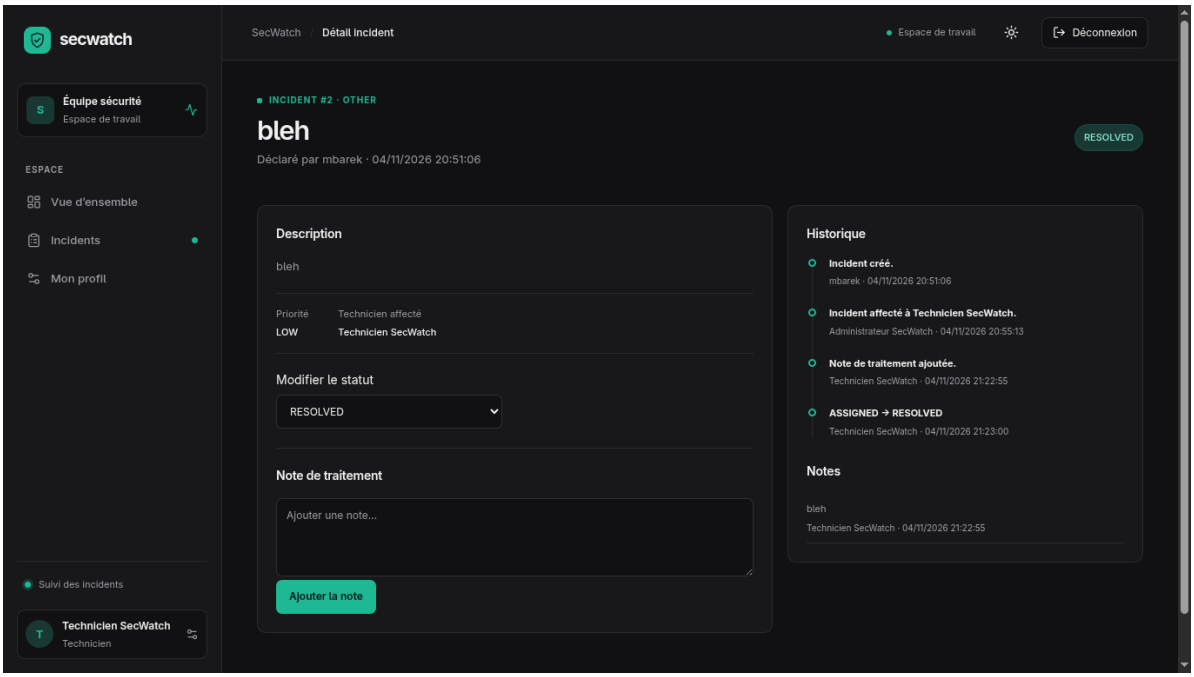


FIGURE 9.8 – Détail et traitement d'un incident

Chapitre 10

Résultats, limites et conclusion

10.1 Résultats

SecWatch met en œuvre l'authentification JWT, les rôles, la protection des routes, le contrôle du propriétaire, la validation, les erreurs HTTP, les profils et le suivi des incidents. L'affectation, les statuts, les notes et l'historique couvrent le parcours de traitement. Les tableaux de bord présentent des informations adaptées à chaque rôle.

10.2 Limites connues

Le Refresh Token, facultatif dans le sujet, n'est pas inclus. Les comptes de démonstration sont réservés au développement local.

10.3 Conclusion

Le projet met en pratique les mécanismes de sécurité d'une application distribuée. Le frontend facilite l'utilisation, mais le backend vérifie le JWT, le rôle et les droits sur chaque incident. La séparation entre contrôleurs, services, repositories et pages React clarifie les responsabilités du système.